

Hybrid Knowledge Graph + RAG for Student-Centric Decision Support

1. Introduction

The goal of this project is to build a **comprehensive institutional AI system** for **Arab Academy for Science, Technology and Maritime Transport (AAST)** capable of understanding, organizing, and reasoning over **all academic and administrative information published on the official website**.

The system is not designed as a generic chatbot. It is designed as a **decision-support system** for real users.

2. Target Users & Objectives

2.1 New Students & Parents

The system helps new students and parents to:

- Choose the most suitable college based on student goals
- Understand:
 - colleges and campuses
 - programs and degrees
 - courses and study plans
 - grading systems
 - tuition fees
 - admission requirements
 - transfer rules from other universities
- Make informed academic and financial decisions

2.2 Current Students

The system supports current students in:

- Course registration decisions
- Identifying remaining graduation requirements

- Understanding prerequisites
- Selecting optimal course combinations
- Planning study schedules
- Organizing study workload
- Interpreting academic bylaws and regulations

The system moves beyond Q&A into **academic guidance**.

3. Data Scope & Coverage

The project explicitly targets **every type of information available on the AAST website**, including:

- Campuses
- Colleges
- Departments
- Programs
- Courses
- Tuition fees
- Grading systems
- Admission requirements
- Transfer regulations
- Academic bylaws
- Staff pages
- PDFs and attachments
- Administrative pages

Any published information is considered part of the knowledge domain.

4. Scraping Issues & Solving Scraping Issues

4.1 Inconsistent Website Architecture

The AAST website is composed of **7+ independent subsystems**, including:

- [contenttemp.php](#) (legacy CMS)
- [programtemp.php](#) (program system)

- Staff CV system ([cv.php](#))
- Presidency website
- Training centers built with React/Vue
- Static HTML templates
- JavaScript-heavy navigation
- Multiple undocumented AJAX APIs
- Joomla/WordPress remnants

Impact:

- No standard URL structure
- No unified schema
- Each college behaves as a mini-website
- Traditional crawlers completely fail

4.2 Dynamic Navigation (No Real HTML Links)

- Menus are JavaScript-driven
- Links appear only after hover/click
- Homepage HTML without JS is almost empty

➡ Crawlers without full browser automation are blind.

4.3 API Instability

Key APIs:

- [getPageContent2021.php](#)
- [getProgramData2021.php](#)
- [getCvNew.php](#)

Observed behavior:

- JSON structure changes
- Null responses
- HTML error pages
- Random redirects

➡ APIs cannot be trusted as a single source.

4.4 HTML Noise

Pages contain:

- 20+ JS files
- Analytics scripts
- Chat widgets
- Repeated headers (3 times)
- Inline JS/CSS
- Hidden DOM blocks

➡ Heavy cleaning and filtering required.

5. Step-by-Step Scraping Solution

Phase 1 — Manual Seed Discovery

- Manual definition of college, admission, training, and service pages
- No reliance on homepage discovery

Phase 2 — Headful Playwright Crawling

- Simulated interactions: hover menus, click dropdowns, scroll pages
- Wait for AJAX completion
- DOM stabilization

Phase 3 — Multi-Endpoint Extraction For each page:

1. Try API
2. Fallback to rendered HTML
3. Trigger AJAX manually
4. Brute-force ID-based discovery

Phase 4 — Cleaning & Deduplication

- URL canonicalization
- Content hashing
- Language normalization
- Header/footer removal

Phase 5 — Normalization All content mapped into structured entities:

- campuses
- colleges
- programs
- courses
- fees
- grading systems
- admission rules

6. Embedding Model Comparison (With Numbers)

6.1 Experimental Setup

- Device: HP Victus
- GPU: RTX 4050
- CPU: Ryzen 7
- RAM: 16GB DDR5
- Languages: Arabic + English

6.2 Small Dataset Results

Model	Avg_Sim_Positive	Avg_Sim_Negative	Time (s)
SBERT	0.450	0.044	0.41
DistilBERT	0.492	0.142	0.09
MiniLM	0.512	0.072	0.08
Multilingual MiniLM	0.577	0.079	0.13
MultiQA MiniLM	0.580	0.244	0.08

6.3 Large Dataset (2000 Samples)

Model	Avg_Sim_Positive	Avg_Sim_Negative	Time (s)
SBERT	0.609	0.075	262.00
DistilBERT	0.598	0.046	143.74
MiniLM	0.600	0.077	140.17
Multilingual MiniLM	0.729	0.068	231.77
MultiQA MiniLM	0.664	0.321	140.62

6.4 Final Selection

Model: paraphrase-multilingual-MiniLM-L12-v2

Reasons:

- Highest semantic similarity
- Strong Arabic–English alignment
- Acceptable inference time
- Fully local & deterministic

7. RAG Pipeline (Step-by-Step)

1. Clean normalized dataset
2. Semantic sentence-aware chunking
3. Multilingual embedding generation
4. Chroma vector database construction
5. Query embedding
6. Top-k semantic retrieval
7. Context filtering
8. Context injection into LLM

8. Why RAG Alone Is Not Enough

RAG failed on structural queries such as:

- How many colleges are in Alamein campus?
- Who is the dean of the College of AI?

Reason:

- RAG retrieves text
- Not facts
- Not relations
- No guarantees

9. Knowledge Graph Construction (KG Build – Raw Data Input)

9.1 Core Insight

Mention ≠ Knowledge

9.2 KG Pipeline

1. Data cleaning
2. Candidate sentence extraction
3. LLM proposes candidate facts
4. Fact / Reject separation
5. Hard semantic validation
6. Neo4j insertion

9.3 Ontology

Entities:

- University
- Campus
- College

- Department
- Program
- Course

Relations:

- HAS_CAMPUS
- HAS_COLLEGE
- HAS_DEPARTMENT
- OFFERS_PROGRAM
- HAS_COURSE
- LOCATED_IN
- PART_OF

9.4 Hard Semantic Validation Rules

Reject facts involving:

- People
- Events
- Announcements
- UI elements
- Facilities
- Generic terms
- External universities (unless PART_OF)

10. Hybrid KG + RAG Architecture

- KG ensures **truth & structure**
- RAG ensures **context & explanation**
- LLM ensures **natural interaction**

They are complementary, not competing.

11. Final System Characteristics

- Large, clean Knowledge Graph

- Multilingual semantic retrieval
- Deterministic pipelines
- Fully local execution
- Decision-support oriented

12. Final Conclusion

This project is not:

- A scraper
- A chatbot
- A simple RAG system